

Question · Logic · Code · Output

Table of Contents

Topic 1: Introduction of Arrays & One-Dimensional Array

5 Easy · 3 Moderate · 2 Very Hard

Topic 2: Programs of One-Dimensional Array

5 Easy · 3 Moderate · 2 Very Hard

Topic 3: Two-Dimensional Array

5 Easy · 3 Moderate · 2 Very Hard

Topic 4: Operations Based on Jagged Arrays

5 Easy · 3 Moderate · 2 Very Hard

Topic 5: Advance Programs Based on Arrays

5 Easy · 3 Moderate · 2 Very Hard

Topic 6: Programs of Multi-Dimensional Array

5 Easy · 3 Moderate · 2 Very Hard

Topic 7: Programs Based on Access Modifiers

5 Easy · 3 Moderate · 2 Very Hard

Topic 8: Programs Based on Run-time Polymorphism

5 Easy · 3 Moderate · 2 Very Hard

TOPIC 1

Introduction of Arrays & One-Dimensional Array

Covers array declaration, traversal, searching and basic manipulation.

**Q0
1**

Best Time to Buy and Sell Stock

Amazon · LeetCode #121

EASY

LOGIC / APPROACH

Track the minimum price seen so far while iterating. At each position compute profit = currentPrice - minPrice. Update maxProfit if this exceeds the current best. Single O(n) pass, O(1) space.

JAVA CODE

```
public class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE, maxProfit = 0;
        for (int price : prices) {
            if (price < minPrice) minPrice = price;
            else if (price - minPrice > maxProfit)
                maxProfit = price - minPrice;
        }
        return maxProfit;
    }
    public static void main(String[] args) {
        Solution s = new Solution();
        System.out.println(s.maxProfit(new int[]{7,1,5,3,6,4}));
        System.out.println(s.maxProfit(new int[]{7,6,4,3,1}));
    }
}
```

OUTPUT

```
Output: 5
Output: 0
```

**Q0
2**

Find Maximum & Minimum in Array

TCS · HackerRank – Arrays

EASY

LOGIC / APPROACH

Initialise max and min to the first element. Iterate once, updating max when a larger value is found and min when a smaller value is found. No sorting needed — O(n) time, O(1) space.

JAVA CODE

```

public class MaxMin {
    static int[] findMaxMin(int[] temp) {
        int max = temp[0], min = temp[0];
        for (int i = 1; i < temp.length; i++) {
            if (temp[i] > max) max = temp[i];
            if (temp[i] < min) min = temp[i];
        }
        return new int[]{max, min};
    }
    public static void main(String[] args) {
        int[] r = findMaxMin(new int[]{3,5,1,9,2});
        System.out.println("max=" + r[0] + ", min=" + r[1]);
    }
}

```

OUTPUT

```
max=9, min=1
```

Q0
3

Reverse an Array In-Place

Infosys · HackerRank – Arrays

EASY

LOGIC / APPROACH

Use two pointers: left starting at index 0 and right at n-1. Swap elements and move both pointers inward until they meet. This achieves $O(n)$ time with $O(1)$ space — no extra array needed.

JAVA CODE

```

public class ReverseArray {
    static void reverse(int[] arr) {
        int left = 0, right = arr.length - 1;
        while (left < right) {
            int tmp = arr[left];
            arr[left++] = arr[right];
            arr[right--] = tmp;
        }
    }
    public static void main(String[] args) {
        int[] songs = {1,2,3,4,5};
        reverse(songs);
        System.out.println(java.util.Arrays.toString(songs));
    }
}

```

OUTPUT

```
[5, 4, 3, 2, 1]
```

Q0
4

Second Largest Element

*Wipro · GeeksForGeeks***EASY**

LOGIC / APPROACH

Track two variables: first (largest) and second (second largest). On each element: if it exceeds first, push first down to second and update first; else if it exceeds second but is not equal to first, update second. If second remains MIN_VALUE at the end, return -1 (all duplicates).

JAVA CODE

```
public class SecondLargest {
    static int findSecond(int[] scores) {
        int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
        for (int s : scores) {
            if (s > first) { second = first; first = s; }
            else if (s > second && s != first) second = s;
        }
        return (second == Integer.MIN_VALUE) ? -1 : second;
    }
    public static void main(String[] args) {
        System.out.println(findSecond(new int[]{12,35,1,10,34,1}));
        System.out.println(findSecond(new int[]{10,10,10}));
    }
}
```

OUTPUT

```
34
-1
```

Q0
5

Move Zeroes to End

*Cognizant · LeetCode #283***EASY**

LOGIC / APPROACH

Use a slow pointer 'pos' to track the next write position for non-zero values. Iterate with a fast pointer; whenever a non-zero element is found, write it to arr[pos] and advance pos. After the loop, fill positions from pos to end with zeros.

JAVA CODE

```
public class MoveZeroes {
    static void moveZeroes(int[] pixels) {
        int pos = 0;
        for (int p : pixels) if (p != 0) pixels[pos++] = p;
        while (pos < pixels.length) pixels[pos++] = 0;
    }
    public static void main(String[] args) {
        int[] px = {0,1,0,3,12};
        moveZeroes(px);
        System.out.println(java.util.Arrays.toString(px));
    }
}
```

OUTPUT

[1, 3, 12, 0, 0]

Q0
6

Maximum Subarray Sum (Kadane's)

Goldman Sachs · LeetCode #53

MODERATE

LOGIC / APPROACH

Kadane's algorithm: maintain a running currentSum. At each index, either start fresh (take nums[i] alone) or extend the previous subarray (currentSum + nums[i]). Track the maximum seen and the start/end indices: reset start when currentSum resets, update end when global max updates.

JAVA CODE

```
public class MaxSubarray {
    static int[] maxSubarray(int[] nums) {
        int maxSum = nums[0], currSum = nums[0];
        int start = 0, end = 0, tempStart = 0;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > currSum + nums[i]) {
                currSum = nums[i]; tempStart = i;
            } else currSum += nums[i];
            if (currSum > maxSum) {
                maxSum = currSum; start = tempStart; end = i;
            }
        }
        return new int[]{maxSum, start, end};
    }
    public static void main(String[] args) {
        int[] r = maxSubarray(new int[]{-2,1,-3,4,-1,2,1,-5,4});
        System.out.println("Sum=" + r[0] + " idx [" + r[1] + "-" + r[2] + "]");
    }
}
```

OUTPUT

Sum=6 idx [3-6]

Q0
7

Find All Duplicates in Array

Adobe · LeetCode #442

MODERATE

LOGIC / APPROACH

Since values are in [1,n], use the array itself as a hash map. For each value v, negate the element at index (|v|-1). If that element is already negative, v has appeared before — add it to the result. Restore signs if needed. O(n) time, O(1) space.

JAVA CODE

```
import java.util.*;
public class FindDuplicates {
    static List<Integer> findDuplicates(int[] nums) {
        List<Integer> res = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) {
            int idx = Math.abs(nums[i]) - 1;
            if (nums[idx] < 0) res.add(idx + 1);
            else nums[idx] = -nums[idx];
        }
        return res;
    }
    public static void main(String[] args) {
        System.out.println(findDuplicates(new int[]{4,3,2,7,8,2,3,1}));
    }
}
```

OUTPUT

[2, 3]

Q0
8

Rotate Array by K Steps

Accenture · LeetCode #189

MODERATE

LOGIC / APPROACH

Three-reversal trick: normalise $k = k \% n$. (1) Reverse entire array. (2) Reverse first k elements. (3) Reverse remaining $n-k$ elements. This achieves right rotation in $O(n)$ time and $O(1)$ space without any extra array.

JAVA CODE

```
public class RotateArray {
    static void reverse(int[] a, int l, int r) {
        while (l < r) { int t=a[l]; a[l++]=a[r]; a[r--]=t; }
    }
    static void rotate(int[] nums, int k) {
        int n = nums.length; k %= n;
        reverse(nums, 0, n-1);
        reverse(nums, 0, k-1);
        reverse(nums, k, n-1);
    }
    public static void main(String[] args) {
        int[] a = {1,2,3,4,5,6,7};
        rotate(a, 3);
        System.out.println(java.util.Arrays.toString(a));
    }
}
```

OUTPUT

[5, 6, 7, 1, 2, 3, 4]

Q0
9

Trapping Rain Water

*Google · LeetCode #42***VERY HARD**

LOGIC / APPROACH

Two-pointer approach: maintain leftMax and rightMax. The side with the smaller max determines how much water that pointer can hold. Water at pointer = max - height[pointer]. Move the smaller-max pointer inward. O(n) time, O(1) space.

JAVA CODE

```
public class TrappingRain {
    public int trap(int[] height) {
        int left=0, right=height.length-1;
        int leftMax=0, rightMax=0, water=0;
        while (left < right) {
            if (height[left] < height[right]) {
                if (height[left] >= leftMax) leftMax = height[left];
                else water += leftMax - height[left];
                left++;
            } else {
                if (height[right] >= rightMax) rightMax = height[right];
                else water += rightMax - height[right];
                right--;
            }
        }
        return water;
    }
    public static void main(String[] args) {
        System.out.println(new TrappingRain().trap(
            new int[]{0,1,0,2,1,0,1,3,2,1,2,1}));
    }
}
```

OUTPUT

6**Q1**
0

Median of Two Sorted Arrays

*Netflix · LeetCode #4***VERY HARD**

LOGIC / APPROACH

Binary search on the smaller array. Partition both arrays so the left halves together have (m+n+1)/2 elements. Check the invariant: maxLeft1 <= minRight2 AND maxLeft2 <= minRight1. Adjust the partition accordingly. The median is derived from boundary elements. O(log min(m,n)).

JAVA CODE


```

public class MedianSortedArrays {
    public double findMedianSortedArrays(int[] A, int[] B) {
        if (A.length > B.length) return findMedianSortedArrays(B, A);
        int m=A.length, n=B.length, lo=0, hi=m;
        while (lo <= hi) {
            int i = (lo+hi)/2, j = (m+n+1)/2 - i;
            int mL1 = (i==0)?Integer.MIN_VALUE:A[i-1];
            int mR1 = (i==m)?Integer.MAX_VALUE:A[i];
            int mL2 = (j==0)?Integer.MIN_VALUE:B[j-1];
            int mR2 = (j==n)?Integer.MAX_VALUE:B[j];
            if (mL1<=mR2 && mL2<=mR1) {
                if ((m+n)%2==0)
                    return (Math.max(mL1,mL2)+Math.min(mR1,mR2))/2.0;
                return Math.max(mL1,mL2);
            } else if (mL1>mR2) hi=i-1; else lo=i+1;
        }
        return -1;
    }
    public static void main(String[] args) {
        MedianSortedArrays sol = new MedianSortedArrays();
        System.out.println(sol.findMedianSortedArrays(
            new int[]{1,3}, new int[]{2}));
        System.out.println(sol.findMedianSortedArrays(
            new int[]{1,2}, new int[]{3,4}));
    }
}

```

OUTPUT

```

2.0
2.5

```

TOPIC 2

Programs of One-Dimensional Array

Practical 1-D array coding exercises from MNC screening and hackathon rounds.

**Q0
1**

Running Sum of 1D Array

Google · LeetCode #1480

EASY

LOGIC / APPROACH

Modify the array in-place: starting from index 1, add the previous element to the current one. This converts each element into a cumulative sum in a single $O(n)$ pass with $O(1)$ extra space.

JAVA CODE

```
public class RunningSum {
    static int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++)
            nums[i] += nums[i-1];
        return nums;
    }
    public static void main(String[] args) {
        System.out.println(java.util.Arrays.toString(
            runningSum(new int[]{1,2,3,4})));
    }
}
```

OUTPUT

```
[1, 3, 6, 10]
```

**Q0
2**

Element Frequency Count

Infosys · HackerRank

EASY

LOGIC / APPROACH

Use a HashMap to count frequencies. Iterate once over the array: for each element, merge with a count of 1 (incrementing existing counts). Print key-value pairs. Handles negatives and any integer range in $O(n)$ time and $O(k)$ space where k = unique elements.

JAVA CODE

```
import java.util.*;
public class FreqCount {
    public static void main(String[] args) {
        int[] items = {4,3,2,4,1,3,4};
        Map<Integer,Integer> freq = new LinkedHashMap<>();
        for (int x : items)
            freq.merge(x, 1, Integer::sum);
        freq.forEach((k,v) -> System.out.println(k + ": " + v));
    }
}
```

OUTPUT

4: 3
3: 2
2: 1
1: 1

Q0
3

Remove Duplicates from Sorted Array

Wipro · LeetCode #26

EASY

LOGIC / APPROACH

Two-pointer technique on a sorted array: slow pointer k marks the next write position. Fast pointer i scans ahead. When $\text{nums}[i] \neq \text{nums}[k-1]$, a new unique value is found — write it to $\text{nums}[k]$ and advance k . Return k as the new length. $O(n)$ time, $O(1)$ space.

JAVA CODE

```
public class RemoveDuplicates {  
    static int removeDuplicates(int[] ids) {  
        if (ids.length == 0) return 0;  
        int k = 1;  
        for (int i = 1; i < ids.length; i++)  
            if (ids[i] != ids[k-1]) ids[k++] = ids[i];  
        return k;  
    }  
    public static void main(String[] args) {  
        int[] ids = {0,0,1,1,1,2,2,3,3,4};  
        int k = removeDuplicates(ids);  
        System.out.println("Length: " + k);  
        System.out.println(java.util.Arrays.toString(  
            java.util.Arrays.copyOf(ids, k)));  
    }  
}
```

OUTPUT

Length: 5
[0, 1, 2, 3, 4]

Q0
4

Check if Array is Sorted

TCS · Classic

EASY

LOGIC / APPROACH

Linear scan with early exit: iterate from index 1 to $n-1$. If any element is less than the previous element, the array is not sorted — return false immediately. If the loop completes without finding such a pair, return true. $O(n)$ time, $O(1)$ space.

JAVA CODE

```

public class IsSorted {
    static boolean isSorted(int[] codes) {
        for (int i = 1; i < codes.length; i++)
            if (codes[i] < codes[i-1]) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isSorted(new int[]{1,2,3,4,5}));
        System.out.println(isSorted(new int[]{1,3,2,4}));
    }
}

```

OUTPUT

```

true
false

```

Q0
5

Left Rotate Array by D Positions

Cognizant · HackerRank

EASY

LOGIC / APPROACH

Normalise $d = d \% n$ to handle $d \geq n$. Three-reversal trick for left rotation: (1) Reverse first d elements, (2) Reverse remaining $n-d$ elements, (3) Reverse the whole array. This moves the first d elements to the end in $O(n)$ time and $O(1)$ space.

JAVA CODE

```

public class LeftRotate {
    static void rev(int[] a, int l, int r) {
        while(l<r){int t=a[l];a[l++]=a[r];a[r--]=t;}
    }
    static void leftRotate(int[] belt, int d) {
        int n = belt.length; d %= n;
        rev(belt, 0, d-1);
        rev(belt, d, n-1);
        rev(belt, 0, n-1);
    }
    public static void main(String[] args) {
        int[] b = {1,2,3,4,5};
        leftRotate(b, 2);
        System.out.println(java.util.Arrays.toString(b));
    }
}

```

OUTPUT

```

[3, 4, 5, 1, 2]

```

Q0
6

Subarray Sum Equals K

Flipkart · LeetCode #560

MODERATE

LOGIC / APPROACH

Prefix sum + HashMap: as you compute running prefix sums, check if (prefixSum - k) has been seen before. If yes, those subarrays sum to k. Store each prefix sum's frequency in the map. Works for negative values. O(n) time and space.

JAVA CODE

```
import java.util.*;

public class SubarraySum {
    static int subarraySum(int[] nums, int k) {
        Map<Integer,Integer> map = new HashMap<>();
        map.put(0, 1);
        int count = 0, prefix = 0;
        for (int n : nums) {
            prefix += n;
            count += map.getDefault(prefix - k, 0);
            map.merge(prefix, 1, Integer::sum);
        }
        return count;
    }

    public static void main(String[] args) {
        System.out.println(subarraySum(new int[]{1,1,1}, 2));
        System.out.println(subarraySum(new int[]{1,2,3}, 3));
    }
}
```

OUTPUT

```
2
2
```

Q0
7

Majority Element (Boyer-Moore)

Amazon · LeetCode #169

MODERATE

LOGIC / APPROACH

Boyer-Moore Voting: maintain a candidate and a count. When count is 0, pick the current element as the new candidate. Increment count if current matches candidate, else decrement. Since a majority always exists (appears > n/2 times), it will be the final candidate. O(n) time, O(1) space.

JAVA CODE

```

public class MajorityElement {
    static int majorityElement(int[] votes) {
        int candidate = votes[0], count = 1;
        for (int i = 1; i < votes.length; i++) {
            if (count == 0) { candidate = votes[i]; count = 1; }
            else if (votes[i] == candidate) count++;
            else count--;
        }
        return candidate;
    }
    public static void main(String[] args) {
        System.out.println(majorityElement(new int[]{3,2,3}));
        System.out.println(majorityElement(new int[]{2,2,1,1,1,2,2}));
    }
}

```

OUTPUT

```

3
2

```

Q0
8

Find the Duplicate Number

Microsoft · LeetCode #287

MODERATE

LOGIC / APPROACH

Floyd's Cycle Detection: treat each value as a pointer to the next index. Since one value is duplicated, a cycle exists. Phase 1: find the intersection point using slow/fast pointers. Phase 2: reset one pointer to index 0; advance both one step at a time — they meet at the cycle entry, which is the duplicate. $O(n)$ time, $O(1)$ space.

JAVA CODE

```

public class FindDuplicate {
    static int findDuplicate(int[] nums) {
        int slow = nums[0], fast = nums[0];
        do {
            slow = nums[slow];
            fast = nums[nums[fast]];
        } while (slow != fast);
        slow = nums[0];
        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }
        return slow;
    }
    public static void main(String[] args) {
        System.out.println(findDuplicate(new int[]{1,3,4,2,2}));
        System.out.println(findDuplicate(new int[]{3,1,3,4,2}));
    }
}

```

OUTPUT

```

2
3

```

Q0
9

Longest Increasing Subsequence

*Netflix · LeetCode #300***VERY HARD**

LOGIC / APPROACH

Patience sorting ($O(n \log n)$): maintain a 'tails' array where `tails[i]` is the smallest tail element of all LIS of length $i+1$. For each number, binary search for its position in tails. Replace or extend. The length of tails at the end is the LIS length.

JAVA CODE

```
import java.util.*;
public class LIS {
    static int lengthOfLIS(int[] nums) {
        int[] tails = new int[nums.length];
        int size = 0;
        for (int n : nums) {
            int lo = 0, hi = size;
            while (lo < hi) {
                int mid = (lo+hi)/2;
                if (tails[mid] < n) lo = mid+1; else hi = mid;
            }
            tails[lo] = n;
            if (lo == size) size++;
        }
        return size;
    }
    public static void main(String[] args) {
        System.out.println(lengthOfLIS(new int[]{10,9,2,5,3,7,101,18}));
        System.out.println(lengthOfLIS(new int[]{0,1,0,3,2,3}));
    }
}
```

OUTPUT

```
4
4
```

Q1
0

Sliding Window Maximum

*Uber · LeetCode #239***VERY HARD**

LOGIC / APPROACH

Monotonic Deque (ArrayDeque): maintain a deque of indices in decreasing order of their values. For each new element, pop from the back any indices whose values are smaller (they can never be a future maximum). Pop from the front if the index has fallen outside the window. The front of the deque is always the maximum. $O(n)$ time.

JAVA CODE

```
import java.util.*;
public class SlidingWindowMax {
    static int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] res = new int[n - k + 1];
        Deque<Integer> dq = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            if (!dq.isEmpty() && dq.peekFirst() < i-k+1)
                dq.pollFirst();
            while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i])
                dq.pollLast();
            dq.offerLast(i);
            if (i >= k-1) res[i-k+1] = nums[dq.peekFirst()];
        }
        return res;
    }
    public static void main(String[] args) {
        System.out.println(java.util.Arrays.toString(
            maxSlidingWindow(new int[]{1,3,-1,-3,5,3,6,7}, 3)));
    }
}
```

OUTPUT

```
[3, 3, 5, 5, 6, 7]
```


TOPIC 3 Two-Dimensional Array

Matrix operations, grid traversal and 2-D DP — core FAANG interview topics.

Q0
1

Transpose a Matrix

Samsung · LeetCode #867

EASY

LOGIC / APPROACH

Create a result matrix of dimensions N x M (swapped). Set $\text{result}[j][i] = \text{matrix}[i][j]$ for all i,j using nested loops. Time and space are both $O(m*n)$.

JAVA CODE

```
public class Transpose {
    static int[][] transpose(int[][] m) {
        int R=m.length, C=m[0].length;
        int[][] res = new int[C][R];
        for(int i=0;i<R;i++) for(int j=0;j<C;j++) res[j][i]=m[i][j];
        return res;
    }
    public static void main(String[] args) {
        int[][] mat = {{1,2,3},{4,5,6},{7,8,9}};
        for(int[] r : transpose(mat))
            System.out.println(java.util.Arrays.toString(r));
    }
}
```

OUTPUT

```
[1, 4, 7]
[2, 5, 8]
[3, 6, 9]
```

Q0
2

Spiral Order Matrix Traversal

TCS · LeetCode #54

EASY

LOGIC / APPROACH

Maintain four boundary pointers: top, bottom, left, right. Repeatedly traverse: right across the top row, down the right column, left across the bottom row, up the left column. After each direction, shrink the corresponding boundary. Continue until all boundaries cross. $O(m*n)$.

JAVA CODE

```
import java.util.*;
public class SpiralMatrix {
    static List<Integer> spiralOrder(int[][] m) {
        List<Integer> res = new ArrayList<>();
        int top=0,bot=m.length-1,left=0,right=m[0].length-1;
        while(top<=bot && left<=right) {
            for(int c=left;c<=right;c++) res.add(m[top][c]); top++;
            for(int r=top;r<=bot;r++) res.add(m[r][right]); right--;
            if(top<=bot){for(int c=right;c>=left;c--) res.add(m[bot][c]); bot--;}
            if(left<=right){for(int r=bot;r>=top;r--) res.add(m[r][left]); left++;}
        }
        return res;
    }
    public static void main(String[] args) {
        int[][] mat={{1,2,3},{4,5,6},{7,8,9}};
        System.out.println(spiralOrder(mat));
    }
}
```

OUTPUT

```
[1, 2, 3, 6, 9, 8, 7, 4, 5]
```

Q0
3

Search in 2D Matrix

Capgemini · LeetCode #74

EASY

LOGIC / APPROACH

Treat the entire M x N matrix as a flattened sorted array of length m*n. Standard binary search: for mid index, row = mid/n and col = mid%n. Compare matrix[row][col] with target and adjust lo/hi. O(log(m*n)).

JAVA CODE

```
public class Search2D {
    static boolean searchMatrix(int[][] mat, int target) {
        int m=mat.length, n=mat[0].length, lo=0, hi=m*n-1;
        while(lo<=hi) {
            int mid=(lo+hi)/2;
            int val=mat[mid/n][mid%n];
            if(val==target) return true;
            else if(val<target) lo=mid+1; else hi=mid-1;
        }
        return false;
    }
    public static void main(String[] args) {
        int[][] m={{1,3,5,7},{10,11,16,20},{23,30,34,60}};
        System.out.println(searchMatrix(m,3));
        System.out.println(searchMatrix(m,13));
    }
}
```

OUTPUT

```
true
false
```

Q0
4

Diagonal Sum of Matrix

Wipro · LeetCode #1572

EASY

LOGIC / APPROACH

Loop i from 0 to $n-1$. Add $\text{mat}[i][i]$ (primary diagonal) and $\text{mat}[i][n-1-i]$ (secondary diagonal). For odd n , the centre element $\text{mat}[n/2][n/2]$ is counted twice — subtract it once at the end. $O(n)$ time.

JAVA CODE

```
public class DiagonalSum {
    static int diagonalSum(int[][] mat) {
        int n=mat.length, sum=0;
        for(int i=0;i<n;i++) sum += mat[i][i] + mat[i][n-1-i];
        if(n%2==1) sum -= mat[n/2][n/2];
        return sum;
    }
    public static void main(String[] args) {
        System.out.println(diagonalSum(new int[][]{{1,2,3},{4,5,6},{7,8,9}}));
    }
}
```

OUTPUT

25

Q0
5

Rotate Matrix 90 Degrees Clockwise

Adobe · LeetCode #48

EASY

LOGIC / APPROACH

Two-step in-place approach: (1) Transpose the matrix (swap $\text{matrix}[i][j]$ with $\text{matrix}[j][i]$ for all $j > i$). (2) Reverse each row. Together these produce a 90-degree clockwise rotation. $O(n^2)$ time, $O(1)$ space.

JAVA CODE

```
public class RotateMatrix {
    static void rotate(int[][] m) {
        int n=m.length;
        // Transpose
        for(int i=0;i<n;i++)
            for(int j=i+1;j<n;j++){int t=m[i][j];m[i][j]=m[j][i];m[j][i]=t;}
        // Reverse each row
        for(int[] row:m){
            int l=0,r=n-1;
            while(l<r){int t=row[l];row[l++]=row[r];row[r--]=t;}
        }
    }
    public static void main(String[] args) {
        int[][] m={{1,2,3},{4,5,6},{7,8,9}};
        rotate(m);
        for(int[] r:m) System.out.println(java.util.Arrays.toString(r));
    }
}
```

OUTPUT

```
[7, 4, 1]
[8, 5, 2]
[9, 6, 3]
```

Q0
6

Set Matrix Zeroes

Amazon · LeetCode #73

MODERATE

LOGIC / APPROACH

Use the first row and first column as sentinel storage. A separate boolean tracks whether the first column itself needs zeroing. First pass: record which rows/cols contain zeros. Second pass: zero out cells using the sentinels. Finally handle the first row and first column. $O(m*n)$ time, $O(1)$ space.

JAVA CODE

```
public class SetZeroes {
    static void setZeroes(int[][] m) {
        int R=m.length, C=m[0].length;
        boolean firstColZero = false;
        for(int i=0;i<R;i++){
            if(m[i][0]==0) firstColZero=true;
            for(int j=1;j<C;j++) if(m[i][j]==0){m[i][0]=0;m[0][j]=0;}
        }
        for(int i=1;i<R;i++) for(int j=1;j<C;j++)
            if(m[i][0]==0||m[0][j]==0) m[i][j]=0;
        if(m[0][0]==0) for(int j=0;j<C;j++) m[0][j]=0;
        if(firstColZero) for(int i=0;i<R;i++) m[i][0]=0;
    }
    public static void main(String[] args) {
        int[][] m={{1,1,1},{1,0,1},{1,1,1}};
        setZeroes(m);
        for(int[] r:m) System.out.println(java.util.Arrays.toString(r));
    }
}
```

OUTPUT

```
[1, 0, 1]
[0, 0, 0]
[1, 0, 1]
```

Q0
7

Flood Fill

Microsoft · LeetCode #733

MODERATE

LOGIC / APPROACH

DFS from the starting cell (sr,sc). Record the original colour. If the original colour already equals the new colour, return immediately (no-op). Recursively visit all 4-directional neighbours that share the original colour and repaint them. $O(m*n)$.

JAVA CODE

```

public class FloodFill {
    static void dfs(int[][] img,int r,int c,int orig,int newC){
        if(r<0||r>=img.length||c<0||c>=img[0].length) return;
        if(img[r][c]!=orig) return;
        img[r][c]=newC;
        dfs(img,r+1,c,orig,newC); dfs(img,r-1,c,orig,newC);
        dfs(img,r,c+1,orig,newC); dfs(img,r,c-1,orig,newC);
    }
    static int[][] floodFill(int[][] image,int sr,int sc,int color){
        int orig=image[sr][sc];
        if(orig!=color) dfs(image,sr,sc,orig,color);
        return image;
    }
    public static void main(String[] args) {
        int[][] img={{1,1,1},{1,1,0},{1,0,1}};
        floodFill(img,1,1,2);
        for(int[] r:img) System.out.println(java.util.Arrays.toString(r));
    }
}

```

OUTPUT

```

[2, 2, 2]
[2, 2, 0]
[2, 0, 1]

```

Q0
8

Number of Islands

Uber · LeetCode #200

MODERATE

LOGIC / APPROACH

DFS scan: iterate every cell. When a '1' is found, increment the island counter and launch a DFS that sinks (marks as '0') all connected land cells. This prevents re-counting. Total time $O(m*n)$ since each cell is visited at most once.

JAVA CODE

```

public class NumberOfIslands {
    static void dfs(char[][] g,int r,int c){
        if(r<0||r>=g.length||c<0||c>=g[0].length||g[r][c]=='0') return;
        g[r][c]='0';
        dfs(g,r+1,c);dfs(g,r-1,c);dfs(g,r,c+1);dfs(g,r,c-1);
    }
    static int numIslands(char[][] grid){
        int count=0;
        for(int i=0;i<grid.length;i++)
            for(int j=0;j<grid[0].length;j++)
                if(grid[i][j]=='1'){count++;dfs(grid,i,j);}
        return count;
    }
    public static void main(String[] args){
        char[][] g={{{'1','1','0','0','0'},{'1','1','0','0','0'},
            {'0','0','1','0','0'},{'0','0','0','1','1'}}};
        System.out.println(numIslands(g[0].length>0?g:g));
    }
}

```

OUTPUT

Q0
9**Largest Rectangle in Histogram (Matrix)**

Microsoft · LeetCode #85

VERY HARD

LOGIC / APPROACH

Build a cumulative histogram row by row: $heights[j] += 1$ if $matrix[i][j] == '1'$, else reset to 0. For each row, apply the 'Largest Rectangle in Histogram' algorithm using a monotonic stack. Track width by computing $(i - stack.peek() - 1)$. Take the global maximum. $O(m*n)$ overall.

JAVA CODE

```
import java.util.*;

public class MaxRectangle {
    static int largestRectangleInHistogram(int[] h){
        Deque<Integer> st=new ArrayDeque<>();
        int max=0, n=h.length;
        for(int i=0;i<=n;i++){
            int cur=(i==n)?0:h[i];
            while(!st.isEmpty()&&h[st.peek()]>cur){
                int height=h[st.pop()];
                int width=st.isEmpty()?i:(i-st.peek()-1);
                max=Math.max(max,height*width);
            }
            st.push(i);
        }
        return max;
    }
    static int maximalRectangle(char[][] matrix){
        if(matrix.length==0) return 0;
        int cols=matrix[0].length, max=0;
        int[] h=new int[cols];
        for(char[] row:matrix){
            for(int j=0;j<cols;j++){
                h[j]=(row[j]=='1')?h[j]+1:0;
            }
            max=Math.max(max,largestRectangleInHistogram(h));
        }
        return max;
    }
    public static void main(String[] args){
        char[][] m={{'1','0','1','0','0'},{'1','0','1','1','1'},
                    {'1','1','1','1','1'},{'1','0','0','1','0'}};
        System.out.println(maximalRectangle(m));
    }
}
```

OUTPUT

Q1
0

Shortest Path in Binary Matrix

Google · LeetCode #1091

VERY HARD

LOGIC / APPROACH

BFS from (0,0) exploring all 8 directions. Use a queue of (row,col) pairs and track distance by level. Mark cells visited by setting them to 1. Return the level count when (n-1,n-1) is reached. Return -1 if queue empties without reaching the target. $O(n^2)$.

JAVA CODE

```
import java.util.*;
public class ShortestPath {
    static int shortestPathBinaryMatrix(int[][] grid){
        int n=grid.length;
        if(grid[0][0]==1||grid[n-1][n-1]==1) return -1;
        int[][] dirs={{-1, -1},{-1,0},{-1,1},{0, -1},{0,1},{1, -1},{1,0},{1,1}};
        Queue<int[]> q=new LinkedList<>();
        q.add(new int[]{0,0,1});
        grid[0][0]=1;
        while(!q.isEmpty()){
            int[] cur=q.poll();
            int r=cur[0],c=cur[1],d=cur[2];
            if(r==n-1&&c==n-1) return d;
            for(int[] dir:dirs){
                int nr=r+dir[0],nc=c+dir[1];
                if(nr>=0&&nr<n&&nc>=0&&nc<n&&grid[nr][nc]==0){
                    grid[nr][nc]=1;
                    q.add(new int[]{nr,nc,d+1});
                }
            }
        }
        return -1;
    }
    public static void main(String[] args){
        System.out.println(shortestPathBinaryMatrix(
            new int[][]{{0,0,0},{1,1,0},{1,1,0}}));
    }
}
```

OUTPUT

4

TOPIC 4

Operations Based on Jagged Arrays

Ragged (variable-length row) arrays — Java-specific feature tested in core OOP rounds.

**Q0
1**

Declare & Print Jagged Array

Infosys · Core Java

EASY

LOGIC / APPROACH

Declare `int[][] arr = new int[n][]`. Then for each row `i`, allocate `arr[i] = new int[i+1]`. Fill each cell `arr[i][j] = i*j` using nested loops. Print row by row. $O(n^2)$ time and space.

JAVA CODE

```
public class JaggedDeclare {
    public static void main(String[] args) {
        int n=3;
        int[][] arr = new int[n][];
        for(int i=0;i<n;i++){
            arr[i]=new int[i+1];
            for(int j=0;j<=i;j++) arr[i][j]=i*j;
        }
        for(int i=0;i<n;i++)
            System.out.println("Row"+i+"="+java.util.Arrays.toString(arr[i]));
    }
}
```

OUTPUT

```
Row0:[0]
Row1:[0, 1]
Row2:[0, 2, 4]
```

**Q0
2**

Row Sum of Jagged Array

TCS · Core Java

EASY

LOGIC / APPROACH

Outer loop iterates over rows. Inner loop iterates from 0 to `arr[i].length-1` (the actual length of that jagged row). Accumulate the sum and store in `result[i]`. $O(\text{total elements})$.

JAVA CODE


```

public class JaggedRowSum {
    static int[] rowSums(int[][] arr) {
        int[] res = new int[arr.length];
        for(int i=0;i<arr.length;i++)
            for(int v:arr[i]) res[i]+=v;
        return res;
    }
    public static void main(String[] args) {
        int[][] arr={{1,2},{3,4,5},{6}};
        System.out.println(java.util.Arrays.toString(rowSums(arr)));
    }
}

```

OUTPUT

```
[3, 12, 6]
```

Q0
3

Pascal's Triangle as Jagged Array

Wipro · LeetCode #118

EASY

LOGIC / APPROACH

Allocate $\text{tri}[i] = \text{new int}[i+1]$ for each row. Set $\text{tri}[i][0] = \text{tri}[i][i] = 1$. For inner cells: $\text{tri}[i][j] = \text{tri}[i-1][j-1] + \text{tri}[i-1][j]$. $O(n^2)$ time and space.

JAVA CODE

```

public class PascalTriangle {
    static int[][] pascal(int n) {
        int[][] tri=new int[n+1][];
        for(int i=0;i<=n;i++){
            tri[i]=new int[i+1];
            tri[i][0]=tri[i][i]=1;
            for(int j=1;j<i;j++) tri[i][j]=tri[i-1][j-1]+tri[i-1][j];
        }
        return tri;
    }
    public static void main(String[] args) {
        for(int[] r:pascal(4))
            System.out.println(java.util.Arrays.toString(r));
    }
}

```

OUTPUT

```

[1]
[1, 1]
[1, 2, 1]
[1, 3, 3, 1]
[1, 4, 6, 4, 1]

```

Q0
4

Find Maximum in Each Row

*Cognizant · Classic***EASY**

LOGIC / APPROACH

Nested loop: outer over rows, inner over arr[i]. Track rowMax = Integer.MIN_VALUE and update. After inner loop, store rowMax in result[] and compare with globalMax tracking position (row, col). O(total elements).

JAVA CODE

```
public class JaggedRowMax {
    public static void main(String[] args) {
        int[][] arr={{3,1},{7,2,5},{4}};
        int gMax=Integer.MIN_VALUE, gR=0, gC=0;
        int[] rowMax=new int[arr.length];
        for(int i=0;i<arr.length;i++){
            rowMax[i]=Integer.MIN_VALUE;
            for(int j=0;j<arr[i].length;j++){
                if(arr[i][j]>rowMax[i]){rowMax[i]=arr[i][j];}
                if(arr[i][j]>gMax){gMax=arr[i][j];gR=i;gC=j;}
            }
        }
        System.out.println(java.util.Arrays.toString(rowMax));
        System.out.println("Overall max="+gMax+" at (" +gR+", "+gC+"");
    }
}
```

OUTPUT

```
[3, 7, 4]
Overall max=7 at (1,0)
```

Q0
5

Flatten Jagged Array

*Accenture · Core Java***EASY**

LOGIC / APPROACH

Compute total length = sum of arr[i].length for all i. Allocate flat[totalLen]. Use an index pointer; for each row, use System.arraycopy or a loop to copy elements. O(total elements).

JAVA CODE

```
public class FlattenJagged {
    static int[] flatten(int[][] arr) {
        int total=0;
        for(int[] r:arr) total+=r.length;
        int[] flat=new int[total], idx=new int[]{0};
        for(int[] r:arr) for(int v:r) flat[idx[0]++]=v;
        return flat;
    }
    public static void main(String[] args) {
        System.out.println(java.util.Arrays.toString(
            flatten(new int[][]{{1,2},{3},{4,5,6}})));
    }
}
```

OUTPUT

```
[1, 2, 3, 4, 5, 6]
```

Q0
6

Merge Sorted Rows of Jagged Array

HCL · Classic

MODERATE

LOGIC / APPROACH

For each row i , perform a standard two-pointer merge of $A[i]$ and $B[i]$ into $C[i]$ of size $A[i].length + B[i].length$. Both input rows are sorted, so the merge runs in $O(|A[i]| + |B[i]|)$ per row. Total time $O(\text{total elements})$.

JAVA CODE

```
import java.util.*;
public class MergeSortedJagged {
    static int[] mergeRows(int[] a, int[] b) {
        int[] c=new int[a.length+b.length];
        int i=0,j=0,k=0;
        while(i<a.length&& j<b.length) c[k++]=(a[i]<=b[j])?a[i++]:b[j++];
        while(i<a.length) c[k++]=a[i++];
        while(j<b.length) c[k++]=b[j++];
        return c;
    }
    public static void main(String[] args) {
        int[][] A={{1,3},{2}}, B={{2,4},{1,5}};
        for(int i=0;i<A.length;i++)
            System.out.println(Arrays.toString(mergeRows(A[i],B[i])));
    }
}
```

OUTPUT

```
[1, 2, 3, 4]
[1, 2, 5]
```

Q0
7

Transpose a Jagged Array

Mphasis · Advanced Java

MODERATE

LOGIC / APPROACH

Find $\text{maxCols} = \text{max row length}$. For column j , the output row length = count of input rows with length $> j$. Allocate each output row accordingly, filling with $\text{input}[i][j]$ where it exists, defaulting to 0 otherwise.

JAVA CODE

```

public class TransposeJagged {
    static int[][] transposeJagged(int[][] arr) {
        int rows=arr.length, maxCols=0;
        for(int[] r:arr) maxCols=Math.max(maxCols,r.length);
        int[][] out=new int[maxCols][];
        for(int j=0;j<maxCols;j++){
            int cnt=0;
            for(int[] r:arr) if(r.length>j) cnt++;
            out[j]=new int[cnt];
            int k=0;
            for(int i=0;i<rows;i++) if(arr[i].length>j) out[j][k++]=arr[i][j];
        }
        return out;
    }
    public static void main(String[] args) {
        for(int[] r:transposeJagged(new int[][]{{1,2,3},{4,5}}))
            System.out.println(java.util.Arrays.toString(r));
    }
}

```

OUTPUT

```

[1, 4]
[2, 5]
[3]

```

Q0
8

Search Element in Jagged Array

L&T; Infotech · Classic

MODERATE

LOGIC / APPROACH

Nested loop: outer over rows *i*, inner over columns *j* (bound by `arr[i].length`). When `arr[i][j] == target`, record the position (*i*, *j*) in a result list. Return all positions. $O(\text{total elements})$.

JAVA CODE

```

import java.util.*;
public class SearchJagged {
    static List<int[]> search(int[][] arr, int target) {
        List<int[]> res=new ArrayList<>();
        for(int i=0;i<arr.length;i++)
            for(int j=0;j<arr[i].length;j++)
                if(arr[i][j]==target) res.add(new int[]{i,j});
        return res;
    }
    public static void main(String[] args) {
        List<int[]> pos=search(new int[][]{{1,2,3},{4,2},{5}},2);
        for(int[] p:pos) System.out.println("(" +p[0]+"," +p[1]+")");
    }
}

```

OUTPUT

```

(0,1)
(1,1)

```

Q0
9

Sparse Matrix Multiplication (CSR)

IBM Research · LeetCode #311

VERY HARD

LOGIC / APPROACH

Skip zero cells in A. For each non-zero $A[i][k]$, iterate non-zero cells in row k of B and accumulate $C[i][j] += A[i][k] * B[k][j]$. Precompute a list of non-zero (col, val) pairs per row of B to avoid repeated scans. This skips most zero multiplications.

JAVA CODE

```
public class SparseMatMul {
    static int[][] multiply(int[][] A, int[][] B) {
        int m=A.length, k=A[0].length, n=B[0].length;
        int[][] C=new int[m][n];
        for(int i=0;i<m;i++)
            for(int p=0;p<k;p++){
                if(A[i][p]==0) continue;
                for(int j=0;j<n;j++)
                    if(B[p][j]!=0) C[i][j]+=A[i][p]*B[p][j];
            }
        return C;
    }
    public static void main(String[] args) {
        int[][] A={{1,0,0},{-1,0,3}};
        int[][] B={{7,0,0},{0,0,0},{0,0,1}};
        for(int[] r:multiply(A,B))
            System.out.println(java.util.Arrays.toString(r));
    }
}
```

OUTPUT

```
[7, 0, 0]
[-7, 0, 3]
```

Q1
0

BFS on Variable-Width Jagged Grid

EA Games · Competitive

VERY HARD

LOGIC / APPROACH

BFS from (0,0). For each cell, explore 4-directional neighbours. Key difference from regular 2D BFS: validate $nc < grid[nr].length$ (column bound per row varies). Track visited with a 2D boolean array. Return BFS level when target (last row's last cell) is reached.

JAVA CODE

```

import java.util.*;
public class BFSJagged {
static int bfs(int[][] grid) {
int rows=grid.length;
int targetR=rows-1, targetC=grid[targetR].length-1;
boolean[][] vis=new boolean[rows][];
for(int i=0;i<rows;i++) vis[i]=new boolean[grid[i].length];
Queue<int[]> q=new LinkedList<>();
q.add(new int[]{0,0,0}); vis[0][0]=true;
int[][] dirs={{0,1},{0,-1},{1,0},{-1,0}};
while(!q.isEmpty()){
int[] cur=q.poll();
int r=cur[0],c=cur[1],d=cur[2];
if(r==targetR&& c==targetC) return d;
for(int[] dr:dirs){
int nr=r+dr[0], nc=c+dr[1];
if(nr>=0&&nr<rows&&nc>=0&&nc<grid[nr].length
&&!vis[nr][nc]&&grid[nr][nc]!=0){
vis[nr][nc]=true; q.add(new int[]{nr,nc,d+1});
}
}
}
return -1;
}
public static void main(String[] args) {
System.out.println(bfs(new int[][]{{0,0,0},{0,1,0,0},{0,0}}));
}
}

```

OUTPUT

4

TOPIC 5

Advance Programs Based on Arrays

High-difficulty algorithms tested in FAANG SDE-2 and product-company competitive rounds.

Q0
1

Find Pivot Index

TCS · LeetCode #724

EASY

LOGIC / APPROACH

Precompute totalSum. Iterate left to right maintaining leftSum. At index i, rightSum = totalSum - leftSum - nums[i]. If leftSum == rightSum, return i. Else add nums[i] to leftSum. O(n) time, O(1) space.

JAVA CODE

```
public class PivotIndex {
    static int pivotIndex(int[] nums) {
        int total=0; for(int n:nums) total+=n;
        int left=0;
        for(int i=0;i<nums.length;i++){
            if(left==total-left-nums[i]) return i;
            left+=nums[i];
        }
        return -1;
    }
    public static void main(String[] args) {
        System.out.println(pivotIndex(new int[]{1,7,3,6,5,6}));
        System.out.println(pivotIndex(new int[]{1,2,3}));
    }
}
```

OUTPUT

```
3
-1
```

Q0
2

Can Place Flowers

Accenture · LeetCode #605

EASY

LOGIC / APPROACH

Greedy left-to-right scan: a flower can be planted at position i if flowerbed[i]==0, the left neighbour is 0 (or i is 0), and the right neighbour is 0 (or i is the last). Plant it in-place (set to 1) and decrement n. If n reaches 0, return true early.

JAVA CODE

```

public class CanPlaceFlowers {
    static boolean canPlaceFlowers(int[] bed, int n) {
        for(int i=0;i<bed.length&& n>0;i++){
            boolean leftOk=(i==0||bed[i-1]==0);
            boolean rightOk=(i==bed.length-1||bed[i+1]==0);
            if(bed[i]==0&&leftOk&&rightOk){bed[i]=1;n--;}
        }
        return n<=0;
    }
    public static void main(String[] args) {
        System.out.println(canPlaceFlowers(new int[]{1,0,0,0,1},1));
        System.out.println(canPlaceFlowers(new int[]{1,0,0,0,1},2));
    }
}

```

OUTPUT

```

true
false

```

Q0
3

Arithmetic Array Validation

Infosys · LeetCode #1502

EASY

LOGIC / APPROACH

Sort the array. Compute the expected common difference $\text{diff} = \text{nums}[1] - \text{nums}[0]$. Check all consecutive pairs: if any pair has a different difference, return false. $O(n \log n)$ due to sorting.

JAVA CODE

```

import java.util.Arrays;
public class ArithmeticCheck {
    static boolean canMakeArithmetic(int[] notes) {
        Arrays.sort(notes);
        int diff=notes[1]-notes[0];
        for(int i=2;i<notes.length;i++)
            if(notes[i]-notes[i-1]!=diff) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(canMakeArithmetic(new int[]{3,5,1}));
        System.out.println(canMakeArithmetic(new int[]{1,2,4}));
    }
}

```

OUTPUT

```

true
false

```


Q0
4

Maximum Average Subarray I

*Wipro · LeetCode #643***EASY**

LOGIC / APPROACH

Sliding window of fixed size k : compute the sum of the first k elements. Slide right by adding $\text{nums}[i]$ and removing $\text{nums}[i-k]$, updating maxSum each step. Return maxSum/k as a double. $O(n)$ time, $O(1)$ space.

JAVA CODE

```
public class MaxAvgSubarray {
    static double findMaxAverage(int[] h, int k) {
        int sum=0;
        for(int i=0;i<k;i++) sum+=h[i];
        int maxSum=sum;
        for(int i=k;i<h.length;i++){
            sum+=h[i]-h[i-k];
            maxSum=Math.max(maxSum, sum);
        }
        return (double)maxSum/k;
    }
    public static void main(String[] args) {
        System.out.println(findMaxAverage(new int[]{1,12, -5, -6,50,3},4));
    }
}
```

OUTPUT

12.75**Q0**
5

Min Operations to Make Non-Decreasing

*Cognizant · HackerRank***EASY**

LOGIC / APPROACH

Greedy left-to-right: if $\text{sizes}[i] < \text{sizes}[i-1]$, the deficit ($\text{sizes}[i-1] - \text{sizes}[i]$) must be added via increment operations. Add that deficit to the operation count and update $\text{sizes}[i]$ to match $\text{sizes}[i-1]$. $O(n)$.

JAVA CODE

```
public class MinOpsNonDecreasing {
    static long minOps(int[] sizes) {
        long ops=0;
        for(int i=1;i<sizes.length;i++){
            if(sizes[i]<sizes[i-1]){
                ops+=sizes[i-1]-sizes[i];
                sizes[i]=sizes[i-1];
            }
        }
        return ops;
    }
    public static void main(String[] args) {
        System.out.println(minOps(new int[]{1,5,2,4,1}));
        System.out.println(minOps(new int[]{1,2,3}));
    }
}
```

OUTPUT

```
7
0
```

Q0
6

Product of Array Except Self

Microsoft · LeetCode #238

MODERATE

LOGIC / APPROACH

Two-pass O(1) space approach: Left pass — $\text{result}[i] = \text{product of all elements to the left of } i$. Right pass — multiply $\text{result}[i]$ by a running suffix product from the right. No division, O(n) time, O(1) extra space (output array not counted).

JAVA CODE

```
public class ProductExceptSelf {
    static int[] productExceptSelf(int[] nums) {
        int n=nums.length;
        int[] res=new int[n];
        res[0]=1;
        for(int i=1;i<n;i++) res[i]=res[i-1]*nums[i-1];
        int right=1;
        for(int i=n-1;i>=0;i--){res[i]*=right; right*=nums[i];}
        return res;
    }
    public static void main(String[] args) {
        System.out.println(java.util.Arrays.toString(
            productExceptSelf(new int[]{1,2,3,4})));
    }
}
```

OUTPUT

```
[24, 12, 8, 6]
```

Q0
7

Jump Game II (Min Jumps)

Oracle · LeetCode #45

MODERATE

LOGIC / APPROACH

Greedy BFS-style: track the current jump's boundary (curEnd) and the farthest reachable index. Iterate; when i reaches curEnd , it means we must jump — increment jumps and set $\text{curEnd} = \text{farthest}$. Stop when $\text{curEnd} \geq \text{last index}$. O(n) time, O(1) space.

JAVA CODE

```

public class JumpGameII {
    static int jump(int[] nums) {
        int jumps=0, curEnd=0, farthest=0;
        for(int i=0;i<nums.length-1;i++){
            farthest=Math.max(farthest,i+nums[i]);
            if(i==curEnd){jumps++;curEnd=farthest;}
        }
        return jumps;
    }
    public static void main(String[] args) {
        System.out.println(jump(new int[]{2,3,1,1,4}));
        System.out.println(jump(new int[]{2,3,0,1,4}));
    }
}

```

OUTPUT

```

2
2

```

Q0
8

Next Greater Element

Zomato · LeetCode #496

MODERATE

LOGIC / APPROACH

Process nums2 with a monotonic decreasing stack. When nums2[i] is larger than the stack top, pop and record: stack_top's next greater element = nums2[i]. Store in a HashMap. For each value in nums1, look up the HashMap. O(n+m) time.

JAVA CODE

```

import java.util.*;
public class NextGreater {
    static int[] nextGreaterElement(int[] nums1, int[] nums2) {
        Map<Integer,Integer> nge=new HashMap<>();
        Deque<Integer> st=new ArrayDeque<>();
        for(int n:nums2){
            while(!st.isEmpty()&&st.peek()<n) nge.put(st.pop(),n);
            st.push(n);
        }
        int[] res=new int[nums1.length];
        for(int i=0;i<nums1.length;i++)
            res[i]=nge.getOrDefault(nums1[i],-1);
        return res;
    }
    public static void main(String[] args) {
        System.out.println(java.util.Arrays.toString(
            nextGreaterElement(new int[]{4,1,2},new int[]{1,3,4,2})));
    }
}

```

OUTPUT

```

[-1, 3, -1]

```

Q0
9

Count Inversions in Array

Jane Street · CodeChef INVCNT

VERY HARD

LOGIC / APPROACH

Modified merge sort: during the merge step, when an element from the right half is placed before remaining left-half elements, those left elements all form inversions with it. Count += (mid - leftPtr + 1). Recursively sort and merge. $O(n \log n)$.

JAVA CODE

```
public class CountInversions {
    static long mergeCount(int[] arr, int l, int mid, int r) {
        int[] left=java.util.Arrays.copyOfRange(arr, l, mid+1);
        int[] right=java.util.Arrays.copyOfRange(arr, mid+1, r+1);
        long count=0; int i=0, j=0, k=l;
        while(i<left.length&&j<right.length){
            if(left[i]<=right[j]) arr[k++]=left[i++];
            else {arr[k++]=right[j++]; count+=(left.length-i);}
        }
        while(i<left.length) arr[k++]=left[i++];
        while(j<right.length) arr[k++]=right[j++];
        return count;
    }
    static long sortCount(int[] arr, int l, int r){
        if(l>=r) return 0;
        int mid=(l+r)/2;
        return sortCount(arr, l, mid)+sortCount(arr, mid+1, r)+mergeCount(arr, l, mid, r);
    }
    public static void main(String[] args) {
        int[] a={2,4,1,3,5};
        System.out.println(sortCount(a,0,a.length-1));
    }
}
```

OUTPUT

3

Q1
0

Minimum Window Subarray

Paytm · LeetCode #76 variant

VERY HARD

LOGIC / APPROACH

Sliding window + frequency HashMap: store required element counts. Expand right pointer, decrementing counts in the 'need' map when a required element is found. When all required elements are satisfied (formed == required.size()), shrink from the left to minimise the window. Track minimum length throughout. $O(n)$.

JAVA CODE

```

import java.util.*;
public class MinWindowSubarray {
    static int minWindow(int[] trans, int[] req) {
        Map<Integer,Integer> need=new HashMap<>(), have=new HashMap<>();
        for(int x:req) need.merge(x,1,Integer::sum);
        int formed=0, required=need.size(), minLen=Integer.MAX_VALUE;
        int l=0;
        for(int r=0;r<trans.length;r++){
            have.merge(trans[r],1,Integer::sum);
            if(need.containsKey(trans[r])&&
            have.get(trans[r]).equals(need.get(trans[r]))) formed++;
            while(formed==required){
                minLen=Math.min(minLen,r-l+1);
                have.merge(trans[l],-1,Integer::sum);
                if(need.containsKey(trans[l])&&
                have.get(trans[l])<need.get(trans[l])) formed--;
                l++;
            }
        }
        return minLen==Integer.MAX_VALUE?-1:minLen;
    }
    public static void main(String[] args) {
        System.out.println(minWindow(new int[]{2,1,3,1,2},new int[]{1,2}));
    }
}

```

OUTPUT

2

TOPIC 6

Programs of Multi-Dimensional Array

3-D and higher-dimensional arrays — tested at Qualcomm, NVIDIA, and scientific computing roles.

Q0
1

3D Array Declaration & Print

L&T; · Core Java

EASY

LOGIC / APPROACH

Declare `int[][][] arr = new int[L][M][N]`. Triple nested loops: `arr[i][j][k] = i+j+k`. Print each layer (i) as a 2D slice using nested j,k loops. $O(L*M*N)$.

JAVA CODE

```
public class ThreeDArray {
    public static void main(String[] args) {
        int L=2,M=2,N=2;
        int[][][] arr=new int[L][M][N];
        for(int i=0;i<L;i++) for(int j=0;j<M;j++) for(int k=0;k<N;k++)
            arr[i][j][k]=i+j+k;
        for(int i=0;i<L;i++){
            System.out.print("Layer "+i+": ");
            for(int j=0;j<M;j++){
                System.out.print("[");
                for(int k=0;k<N;k++)
                    System.out.print(arr[i][j][k]+(k<N-1?" ":""));
                System.out.print("]" + (j<M-1?" ":""));
            }
            System.out.println();
        }
    }
}
```

OUTPUT

```
Layer 0: [0,1],[1,2]
Layer 1: [1,2],[2,3]
```

Q0
2

Sum of Two 3D Tensors

IBM · Core Java

EASY

LOGIC / APPROACH

Triple nested loop: `C[i][j][k] = A[i][j][k] + B[i][j][k]` for all valid indices. Both tensors must have the same shape $L \times M \times N$. $O(L*M*N)$ time and space.

JAVA CODE

```

public class TensorAdd {
    static int[][][] add3D(int[][][] A, int[][][] B) {
        int L=A.length, M=A[0].length, N=A[0][0].length;
        int[][][] C=new int[L][M][N];
        for(int i=0;i<L;i++) for(int j=0;j<M;j++) for(int k=0;k<N;k++)
            C[i][j][k]=A[i][j][k]+B[i][j][k];
        return C;
    }
    public static void main(String[] args) {
        int[][][] A={{1,2}}, B={{3,4}};
        int[][][] C=add3D(A,B);
        System.out.println(""+C[0][0][0]+","+C[0][0][1]+"");
    }
}

```

OUTPUT

[4, 6]

Q0
3

Average Intensity Per Frame (3D Video)

Adobe · Core Java

EASY

LOGIC / APPROACH

For each frame i , accumulate the sum of all pixel values across rows and columns. Divide by $\text{rows} \times \text{cols}$ to get the average. Return as $\text{double}[\text{frames}]$. $O(\text{frames} \times \text{rows} \times \text{cols})$.

JAVA CODE

```

public class FrameAvg {
    static double[] avgPerFrame(int[][][] video) {
        int frames=video.length, R=video[0].length, C=video[0][0].length;
        double[] avg=new double[frames];
        for(int i=0;i<frames;i++){
            long sum=0;
            for(int[] row:video[i]) for(int v:row) sum+=v;
            avg[i]=(double)sum/(R*C);
        }
        return avg;
    }
    public static void main(String[] args) {
        int[][][] v={{1,3},{5,7}},{2,4},{6,8}};
        System.out.println(java.util.Arrays.toString(avgPerFrame(v)));
    }
}

```

OUTPUT

[4.0, 5.0]

Q0
4

Count Non-Zero Elements in 3D Array

*Qualcomm · Core Java***EASY**

LOGIC / APPROACH

Triple nested loop over all (i,j,k). If `arr[i][j][k] != 0`, increment count and record the position in a list. Print count and all positions. $O(L*M*N)$.

JAVA CODE

```
import java.util.*;
public class NonZero3D {
    public static void main(String[] args) {
        int[][][] arr={{1,0},{0,2}},{3,0},{0,0}};
        int count=0;
        List<String> pos=new ArrayList<>();
        for(int i=0;i<arr.length;i++)
            for(int j=0;j<arr[i].length;j++)
                for(int k=0;k<arr[i][j].length;k++)
                    if(arr[i][j][k]!=0){count++;pos.add("(" +i+", "+j+", "+k+")");}
        System.out.println("Count="+count+", positions:"+pos);
    }
}
```

OUTPUT

```
Count=3, positions:[(0,0,0), (0,1,1), (1,0,0)]
```

Q0
5

Max Element in 3D Array

*Samsung · Core Java***EASY**

LOGIC / APPROACH

Track `globalMax = Integer.MIN_VALUE` and its position (li, lj, lk). Triple nested loop: whenever `arr[i][j][k] > globalMax`, update `globalMax` and position. $O(L*M*N)$.

JAVA CODE

```
public class Max3D {
    public static void main(String[] args) {
        int[][][] arr={{1,9},{2,3}},{5,4},{7,6}};
        int max=Integer.MIN_VALUE, ri=0,rj=0,rk=0;
        for(int i=0;i<arr.length;i++)
            for(int j=0;j<arr[i].length;j++)
                for(int k=0;k<arr[i][j].length;k++)
                    if(arr[i][j][k]>max){max=arr[i][j][k];ri=i;rj=j;rk=k;}
        System.out.println("max="+max+" at (" +ri+", "+rj+", "+rk+")");
    }
}
```

OUTPUT

```
max=9 at (0,0,1)
```


Q0
6

3D Heat Diffusion Simulation

NVIDIA · Scientific Computing

MODERATE

LOGIC / APPROACH

Allocate a copy array for new values. For each interior cell, average itself with all existing axis-aligned neighbours (clamped at boundaries by only including in-bounds neighbours). Use the copy for output to avoid using updated values during the same step. $O(L*M*N)$.

JAVA CODE

```
public class HeatDiffusion {
    static double[][][] diffuse(double[][][] t) {
        int L=t.length,M=t[0].length,N=t[0][0].length;
        double[][][] next=new double[L][M][N];
        int[][] d={{1,0,0},{-1,0,0},{0,1,0},{0,-1,0},{0,0,1},{0,0,-1}};
        for(int i=0;i<L;i++) for(int j=0;j<M;j++) for(int k=0;k<N;k++){
            double sum=t[i][j][k]; int cnt=1;
            for(int[] dir:d){
                int ni=i+dir[0],nj=j+dir[1],nk=k+dir[2];
                if(ni>=0&&ni<L&&nj>=0&&nj<M&&nk>=0&&nk<N){sum+=t[ni][nj][nk];cnt++;}
            }
            next[i][j][k]=sum/cnt;
        }
        return next;
    }
    public static void main(String[] args) {
        double[][][] g={{50.0}};
        System.out.println(diffuse(g)[0][0][0]);
    }
}
```

OUTPUT

50.0

Q0
7

3D Prefix Sum for Volume Queries

Intel · Competitive Programming

MODERATE

LOGIC / APPROACH

Build $pre[i][j][k]$ using the 3D inclusion-exclusion formula. For a range query $(x1,y1,z1) \rightarrow (x2,y2,z2)$, apply 3D inclusion-exclusion with 8 corners to get the sub-cuboid sum in $O(1)$. Build is $O(L*M*N)$, each query $O(1)$.

JAVA CODE

```

public class PrefixSum3D {
    static long[][][] build(int[][][] a) {
        int L=a.length,M=a[0].length,N=a[0][0].length;
        long[][][] p=new long[L+1][M+1][N+1];
        for(int i=1;i<=L;i++) for(int j=1;j<=M;j++) for(int k=1;k<=N;k++)
            p[i][j][k]=a[i-1][j-1][k-1]+p[i-1][j][k]+p[i][j-1][k]+p[i][j][k-1]
            -p[i-1][j-1][k]-p[i-1][j][k-1]-p[i][j-1][k-1]+p[i-1][j-1][k-1];
        return p;
    }
    static long query(long[][][] p,int x1,int y1,int z1,int x2,int y2,int z2){
        x1++;y1++;z1++;x2++;y2++;z2++;
        return p[x2][y2][z2]-p[x1-1][y2][z2]-p[x2][y1-1][z2]-p[x2][y2][z1-1]
            +p[x1-1][y1-1][z2]+p[x1-1][y2][z1-1]+p[x2][y1-1][z1-1]-p[x1-1][y1-1][z1-1];
    }
    public static void main(String[] args) {
        int[][][] a={{1,2},{3,4}},{5,6},{7,8}};
        long[][][] p=build(a);
        System.out.println(query(p,0,0,0,1,1,1));
    }
}

```

OUTPUT

36

Q0
8

3D BFS — Shortest Path in Obstacle Grid

Amazon Robotics · LeetCode variant

MODERATE

LOGIC / APPROACH

BFS with a queue of $\text{int}[3] = \{\text{layer, row, col}\}$. 6-directional movement using direction vectors for ± 1 in each axis. Use a 3D visited boolean array. When the front of the queue equals $(L-1, M-1, N-1)$, return the distance. Return -1 if unreachable. $O(L*M*N)$.

JAVA CODE

```

import java.util.*;
public class BFS3D {
    static int bfs3D(int[][][] g) {
        int L=g.length,M=g[0].length,N=g[0][0].length;
        if(g[0][0][0]==1||g[L-1][M-1][N-1]==1) return -1;
        int[][] dirs={{1,0,0},{-1,0,0},{0,1,0},{0,-1,0},{0,0,1},{0,0,-1}};
        boolean[][][] vis=new boolean[L][M][N];
        Queue<int[]> q=new LinkedList<>();
        q.add(new int[]{0,0,0,0}); vis[0][0][0]=true;
        while(!q.isEmpty()){
            int[] cur=q.poll();
            int i=cur[0],j=cur[1],k=cur[2],d=cur[3];
            if(i==L-1&&j==M-1&&k==N-1) return d;
            for(int[] dir:dirs){
                int ni=i+dir[0],nj=j+dir[1],nk=k+dir[2];
                if(ni>=0&&ni<L&&nj>=0&&nj<M&&nk>=0&&nk<N
                    &&!vis[ni][nj][nk]&&g[ni][nj][nk]==0){
                    vis[ni][nj][nk]=true; q.add(new int[]{ni,nj,nk,d+1});
                }
            }
        }
        return -1;
    }
    public static void main(String[] args) {
        int[][][] grid=new int[2][2][2]; // all zeros
        System.out.println(bfs3D(grid));
    }
}

```

OUTPUT

3

Q0
9

Matrix Chain Multiplication (3D Slices)

Google Brain · CLRS Classic

VERY HARD

LOGIC / APPROACH

Interval DP: $dp[i][j]$ = minimum scalar multiplications to compute the product of matrices i through j . For each chain length, try every split point k : $dp[i][j] = \min(dp[i][k] + dp[k+1][j] + \text{dims}[i-1]*\text{dims}[k]*\text{dims}[j])$. Fill diagonals bottom-up. $O(n^3)$.

JAVA CODE

```

public class MatrixChainMul {
    static int mcm(int[] dims) {
        int n=dims.length-1;
        int[][] dp=new int[n+1][n+1];
        for(int len=2;len<=n;len++){
            for(int i=1;i<=n-len+1;i++){
                int j=i+len-1;
                dp[i][j]=Integer.MAX_VALUE;
                for(int k=i;k<j;k++){
                    int cost=dp[i][k]+dp[k+1][j]+dims[i-1]*dims[k]*dims[j];
                    if(cost<dp[i][j]) dp[i][j]=cost;
                }
            }
        }
        return dp[1][n];
    }
    public static void main(String[] args) {
        System.out.println(mcm(new int[]{10,30,5,60}));
        System.out.println(mcm(new int[]{40,20,30}));
    }
}

```

OUTPUT

```

4500
24000

```

Q1
0

4D Hypercube Flood Fill (Connected Components)

Research · Competitive

VERY HARD

LOGIC / APPROACH

BFS with `int[4]` state. Maintain an 8-directional set: for each of the 4 axes, apply ± 1 . Use a 4D boolean visited array. For each unvisited true cell, increment component count and BFS to mark the entire connected component visited. $O(N^4)$.

JAVA CODE

```

import java.util.*;
public class HyperCubeFloodFill {
    static int components(boolean[][][] g, int N) {
        boolean[][][] vis=new boolean[N][N][N];
        int count=0;
        for(int a=0;a<N;a++) for(int b=0;b<N;b++)
        for(int c=0;c<N;c++) for(int d=0;d<N;d++)
        if(g[a][b][c][d]&&!vis[a][b][c][d]){
            count++;
            Queue<int[]> q=new LinkedList<>();
            q.add(new int[]{a,b,c,d}); vis[a][b][c][d]=true;
            int[][] dirs={{1,0,0,0},{-1,0,0,0},{0,1,0,0},{0,-1,0,0},
            {0,0,1,0},{0,0,-1,0},{0,0,0,1},{0,0,0,-1}};
            while(!q.isEmpty()){
                int[] cur=q.poll();
                for(int[] dir:dirs){
                    int[] n={cur[0]+dir[0],cur[1]+dir[1],cur[2]+dir[2],cur[3]+dir[3]};
                    if(n[0]>=0&&n[0]<N&&n[1]>=0&&n[1]<N&&n[2]>=0&&n[2]<N
                    &&n[3]>=0&&n[3]<N&&g[n[0]][n[1]][n[2]][n[3]]&&!vis[n[0]][n[1]][n[2]][n[3]]){
                        vis[n[0]][n[1]][n[2]][n[3]]=true; q.add(n);
                    }
                }
            }
            return count;
        }
    }
    public static void main(String[] args) {
        boolean[][][] g=new boolean[2][2][2];
        for(boolean[][] l:g) for(boolean[] m:l) for(boolean[] r:m) java.util.Arrays.fill(r,true);
        System.out.println(components(g,2));
    }
}

```

OUTPUT

1

TOPIC 7

Programs Based on Access Modifiers

Java encapsulation (public/private/protected/default) — core OOP round at every MNC.

**Q0
1**

Bank Account Encapsulation

Infosys · Core Java OOP

EASY

LOGIC / APPROACH

Declare private double balance. Public deposit() validates amount > 0, then adds. Public withdraw() validates amount > 0 and amount <= balance (throws InsufficientFundsException otherwise). getBalance() returns the current balance. Custom exception extends RuntimeException.

JAVA CODE

```
class InsufficientFundsException extends RuntimeException {
    InsufficientFundsException(String m){super(m);}
}
class BankAccount {
    private double balance = 0;
    public void deposit(double amt) {
        if(amt<=0) throw new IllegalArgumentException("Invalid amount");
        balance+=amt;
    }
    public void withdraw(double amt) {
        if(amt<=0) throw new IllegalArgumentException("Invalid amount");
        if(amt>balance) throw new InsufficientFundsException("Overdraft!");
        balance-=amt;
    }
    public double getBalance(){ return balance; }
}
public class BankDemo {
    public static void main(String[] args) {
        BankAccount acc=new BankAccount();
        acc.deposit(500); acc.withdraw(200);
        System.out.println(acc.getBalance());
        try{ acc.withdraw(600); }
        catch(InsufficientFundsException e){ System.out.println(e.getMessage()); }
    }
}
```

OUTPUT

```
300.0
Overdraft!
```

Q0
2

Package-Private Calculator

TCS · Core Java OOP

EASY

LOGIC / APPROACH

In the calculator class, declare add() and subtract() as public. The multiply() helper has no access modifier (default/package-private). A class in the same package can call all three; a class in a different package gets a compile error when calling multiply().

JAVA CODE

```
// Package: com.example.math
package com.example.math;
public class Calculator {
    public int add(int a, int b){ return a+b; }
    public int subtract(int a, int b){ return a-b; }
    int multiply(int a, int b){ return a*b; } // package-private
}
// Same package -- works fine
class SamePackageUser {
    void test(){
        Calculator c=new Calculator();
        System.out.println(c.add(5,3));
        System.out.println(c.multiply(5,3)); // OK
    }
}
// Different package -- multiply() call causes compile error
// package com.other; import com.example.math.Calculator;
// c.multiply(5,3); // ERROR: multiply() not visible
```

OUTPUT

```
add(5,3) = 8
multiply from diff package = Compile Error
```

Q0
3

Vehicle Hierarchy with Protected Speed

Wipro · Core Java OOP

EASY

LOGIC / APPROACH

In package A, Vehicle has protected int speed. In package B, Car extends Vehicle and can access speed directly in its overridden accelerate(). An unrelated class in package B cannot access speed — it gets a compile error. Protected means visible to subclasses anywhere and same-package classes.

JAVA CODE

```
// Package A
package pkgA;
public class Vehicle {
protected int speed = 60;
public void accelerate(){ System.out.println("Vehicle accelerating at "+speed); }
}
// Package B
package pkgB;
import pkgA.Vehicle;
public class Car extends Vehicle {
@Override
public void accelerate(){ speed=120; System.out.println("Car accelerating at "+speed); }
}
public class Demo {
public static void main(String[] args){
new Car().accelerate();
// new Vehicle().speed -- compile error from unrelated class
}
}
```

OUTPUT

```
Car accelerating at 120
```

Q0
4

Singleton with Private Constructor

HCL · Design Pattern

EASY

LOGIC / APPROACH

Private constructor prevents direct instantiation. A private static volatile instance variable stores the singleton. getInstance() uses double-checked locking (synchronized block with two null checks) to ensure thread safety. volatile prevents instruction reordering on multi-core CPUs.

JAVA CODE

```
public class ConfigManager {
private static volatile ConfigManager instance;
private ConfigManager() {} // prevent instantiation
public static ConfigManager getInstance() {
if(instance==null){
synchronized(ConfigManager.class){
if(instance==null) instance=new ConfigManager();
}
}
return instance;
}
public void show(){ System.out.println("ConfigManager: "+this.hashCode()); }
public static void main(String[] args) {
ConfigManager a=ConfigManager.getInstance();
ConfigManager b=ConfigManager.getInstance();
System.out.println(a==b);
a.show(); b.show();
}
}
```

OUTPUT


```
true
ConfigManager: <same hashCode>
ConfigManager: <same hashCode>
```

Q0
5

Immutable Class with Private Setters

Accenture · Core Java OOP

EASY

LOGIC / APPROACH

Declare the class final so it cannot be subclassed. Make all fields private final — they can only be set once, in the constructor. Provide no setters. Only getters are public. For mutable fields (like arrays), return defensive copies in getters.

JAVA CODE

```
public final class Transaction {
    private final int id;
    private final double amount;
    private final String type;
    public Transaction(int id, double amount, String type){
        this.id=id; this.amount=amount; this.type=type;
    }
    public int getId(){ return id; }
    public double getAmount(){ return amount; }
    public String getType(){ return type; }
    public static void main(String[] args){
        Transaction t=new Transaction(101,500.0,"CREDIT");
        System.out.println(t.getId()+" "+t.getAmount()+" "+t.getType());
        // t.amount = 100; // Compile error: no setter, field is private final
    }
}
```

OUTPUT

```
101 500.0 CREDIT
```

Q0
6

Builder Pattern with Private Setters

Flipkart · Effective Java

MODERATE

LOGIC / APPROACH

Product has all private fields. The static inner class Builder can set them because inner classes have access to outer class private members. Builder methods return 'this' for chaining. The build() method validates required fields (throws if missing) before constructing the Product via a private constructor.

JAVA CODE

```

public class Product {
    private final String name;
    private final double price;
    private final String category;
    private Product(Builder b){ name=b.name; price=b.price; category=b.category; }
    public static class Builder {
        private String name, category;
        private double price;
        public Builder setName(String n){ name=n; return this; }
        public Builder setPrice(double p){ price=p; return this; }
        public Builder setCategory(String c){ category=c; return this; }
        public Product build(){
            if(name==null) throw new IllegalStateException("Name required");
            return new Product(this);
        }
    }
    public String toString(){ return name+"/"+price+"/"+category; }
    public static void main(String[] args){
        Product p=new
        Product.Builder().setName("Phone").setPrice(999).setCategory("Electronics").build();
        System.out.println(p);
    }
}

```

OUTPUT

```
Phone/999.0/Electronics
```

Q0
7

Library System — Package Encapsulation

Cognizant · OOP Design

MODERATE

LOGIC / APPROACH

Public API in package library.api: only borrow() and return() are public. Implementation details (isAvailable, updateInventory) live in library.internal with no access modifier (package-private). Client code imports only library.api — any attempt to call internal methods triggers a compile error.

JAVA CODE

```
// package library.api
package library.api;
import library.internal.Inventory;
public class Library {
    private Inventory inv = new Inventory();
    public String borrow(int bookId){
        if(inv.isAvailable(bookId)){inv.updateInventory(bookId,-1);return "Borrowed";}
        return "Not available";
    }
    public String returnBook(int bookId){inv.updateInventory(bookId,1);return "Returned";}
}
// package library.internal
package library.internal;
import java.util.*;
public class Inventory {
    private Map<Integer,Integer> stock=new HashMap<>();
    Inventory(){ stock.put(1,3); } // default access
    boolean isAvailable(int id){ return stock.getOrDefault(id,0)>0; }
    void updateInventory(int id, int delta){ stock.merge(id,delta,Integer::sum); }
}
// Client (different package)
// library.api.Library lib = new library.api.Library();
// System.out.println(lib.borrow(1)); // Works
// new library.internal.Inventory().isAvailable(1); // Compile error
```

OUTPUT

```
Book borrowed
// Direct internal access = Compile Error
```

Q0
8

Reflection to Access Private Fields

Oracle · Java Advanced

MODERATE

LOGIC / APPROACH

Use `Class.getDeclaredField(name)` to get a `Field` object for a private field. Call `f.setAccessible(true)` to bypass access control at runtime. `f.get(obj)` reads the value; `f.set(obj, newValue)` modifies it. Wrap in try-catch for `IllegalAccessException`. Java 17+ with modules may block this unless module is opened.

JAVA CODE

```
import java.lang.reflect.*;
class SecretBox { private int secret = 42; }
public class ReflectionDemo {
    public static void main(String[] args) throws Exception {
        SecretBox box = new SecretBox();
        Field f = box.getClass().getDeclaredField("secret");
        f.setAccessible(true);
        System.out.println("Before: " + f.get(box));
        f.set(box, 99);
        System.out.println("After: " + f.get(box));
    }
}
```

OUTPUT

Before: 42

After: 99

Q0
9

Plugin System with ServiceLoader

JetBrains · Java Advanced

VERY HARD

LOGIC / APPROACH

Define a public Plugin interface in the api package. Implement it with a package-private class in the impl package — clients cannot instantiate it directly. Register the implementation in META-INF/services/api.Plugin. A PluginFactory uses ServiceLoader.load(Plugin.class) to discover and return implementations through the public interface only.

JAVA CODE

```
// api/Plugin.java
package api;
public interface Plugin { void execute(); }
// impl/SpellCheckPlugin.java
package impl;
import api.Plugin;
class SpellCheckPlugin implements Plugin { // package-private
    public void execute(){ System.out.println("SpellCheck running"); }
}
// META-INF/services/api.Plugin contains: impl.SpellCheckPlugin
// api/PluginFactory.java
package api;
import java.util.ServiceLoader;
public class PluginFactory {
    public static Plugin load() {
        return ServiceLoader.load(Plugin.class).findFirst().orElseThrow();
    }
}
public class Main {
    public static void main(String[] args){
        api.Plugin p = api.PluginFactory.load();
        p.execute();
        // new impl.SpellCheckPlugin(); // Compile error -- package-private
    }
}
```

OUTPUT

```
SpellCheck running
// Direct instantiation = Compile Error
```

LOGIC / APPROACH

module-info.java controls package visibility at the module level. 'exports pkg to module' = qualified export; only the named module can access that package. 'opens pkg to module' = allows reflective access only to the named module. All other packages are hidden by default. This enforces encapsulation even against reflection attacks.

JAVA CODE

```
// oracle.sdk/module-info.java
module oracle.sdk {
  exports com.oracle.api to com.partner; // only partner can use API
  opens com.oracle.internal to framework; // framework can reflect internal
  // com.oracle.secret stays completely hidden
}
// com.partner/module-info.java
module com.partner {
  requires oracle.sdk;
}
// com.partner code can use com.oracle.api
// com.oracle.internal is inaccessible to com.partner
// Any untrusted module gets InaccessibleObjectException
// framework module gets reflective access to com.oracle.internal only
// Usage in partner code:
// import com.oracle.api.OracleService;
// OracleService svc = new OracleService(); // Works
// import com.oracle.internal.InternalHelper;
// InternalHelper h = new InternalHelper(); // Module error!
```

OUTPUT

```
Qualified export enforces access boundary
Untrusted access -> InaccessibleObjectException
```

TOPIC 8

Programs Based on Run-time Polymorphism

Method overriding, dynamic dispatch and design patterns — tested at FAANG and product MNCs.

Q0
1

Animal Sound — Dynamic Dispatch

TCS · Core Java OOP

EASY

LOGIC / APPROACH

Create an Animal base class with a virtual makeSound() method. Each subclass (Dog, Cat, Cow) overrides it. When stored in an Animal[] array and called polymorphically, the JVM resolves the actual method at runtime using the object's vtable — not the reference type. This is called dynamic dispatch.

JAVA CODE

```
class Animal { public void makeSound(){ System.out.println("..."); } }
class Dog extends Animal { @Override public void makeSound(){ System.out.println("Woof!"); } }
class Cat extends Animal { @Override public void makeSound(){ System.out.println("Meow!"); } }
class Cow extends Animal { @Override public void makeSound(){ System.out.println("Moo!"); } }
public class ZooDemo {
    public static void main(String[] args) {
        Animal[] zoo = { new Dog(), new Cat(), new Cow() };
        for(Animal a : zoo) a.makeSound();
    }
}
```

OUTPUT

```
Woof!
Meow!
Moo!
```

Q0
2

Shape Area Calculator

Wipro · Core Java OOP

EASY

LOGIC / APPROACH

Abstract class Shape declares abstract double area(). Each concrete subclass provides its own formula: Circle uses πr^2 , Rectangle uses $w \cdot h$, Triangle uses $0.5 \cdot \text{base} \cdot \text{height}$. Storing all shapes in a Shape[] and calling area() polymorphically lets the JVM dispatch to the right formula at runtime.

JAVA CODE

```

abstract class Shape { abstract double area(); }
class Circle extends Shape {
double r; Circle(double r){this.r=r;}
@Override public double area(){ return Math.PI*r*r; }
}
class Rectangle extends Shape {
double w,h; Rectangle(double w,double h){this.w=w;this.h=h;}
@Override public double area(){ return w*h; }
}
class Triangle extends Shape {
double b,h; Triangle(double b,double h){this.b=b;this.h=h;}
@Override public double area(){ return 0.5*b*h; }
}
public class ShapeDemo {
public static void main(String[] args){
Shape[] shapes={new Circle(5),new Rectangle(4,6),new Triangle(3,4)};
double total=0;
for(Shape s:shapes){ System.out.printf("%.2f%n",s.area()); total+=s.area(); }
System.out.printf("Total: %.2f%n",total);
}
}

```

OUTPUT

```

78.54
24.00
6.00
Total: 108.54

```

Q0
3

Payment Processing System

Paytm · OOP Design

EASY

LOGIC / APPROACH

Abstract class Payment declares processPayment(double). Each subclass (CreditCard, UPI, NetBanking) overrides it with its own logic. Client code holds a Payment reference without knowing the concrete type. This follows the Open-Closed Principle: add new payment types without modifying existing code.

JAVA CODE

```

abstract class Payment { abstract void processPayment(double amt); }
class CreditCard extends Payment {
public void processPayment(double amt){ System.out.println("CreditCard: "+amt); }
}
class UPI extends Payment {
public void processPayment(double amt){ System.out.println("UPI: "+amt); }
}
class NetBanking extends Payment {
public void processPayment(double amt){ System.out.println("NetBanking: "+amt); }
}
public class PaymentDemo {
public static void main(String[] args){
Payment[] gateway={new CreditCard(),new UPI(),new NetBanking()};
for(Payment p:gateway) p.processPayment(100.0);
}
}

```

OUTPUT

```
CreditCard: 100.0
UPI: 100.0
NetBanking: 100.0
```

Q0
4

Notification System — Interface Polymorphism

Infosys · OOP Design

EASY

LOGIC / APPROACH

Interface Notification declares send(String). EmailNotification, SMSNotification, and PushNotification each implement it. NotificationService.sendAll() accepts List and simply calls n.send(msg) for each — it is completely unaware of concrete types. Adding a new channel only requires a new implementing class.

JAVA CODE

```
import java.util.*;
interface Notification { void send(String message); }
class EmailNotification implements Notification {
    public void send(String m){ System.out.println("Email: "+m); }
}
class SMSNotification implements Notification {
    public void send(String m){ System.out.println("SMS: "+m); }
}
class PushNotification implements Notification {
    public void send(String m){ System.out.println("Push: "+m); }
}
class NotificationService {
    void sendAll(List<Notification> list, String msg){
        list.forEach(n->n.send(msg));
    }
}
public class NotifDemo {
    public static void main(String[] args){
        new NotificationService().sendAll(
            List.of(new EmailNotification(),new SMSNotification(),new PushNotification()),
            "Order shipped");
    }
}
```

OUTPUT

```
Email: Order shipped
SMS: Order shipped
Push: Order shipped
```


Q0
5

Tax Calculator — Runtime Type Selection

*Cognizant · OOP Design***EASY**

LOGIC / APPROACH

Abstract `TaxCalculator` declares `computeTax(double price)`. Each subclass stores its own rate as a constant and returns `price * rate`. `Product` holds a `TaxCalculator` reference. The correct subclass is selected when the product is created, and `computeTax()` resolves at runtime — no switch/if on type needed.

JAVA CODE

```
abstract class TaxCalculator { abstract double computeTax(double price); }
class FoodTax extends TaxCalculator { public double computeTax(double p){return p*0.05;} }
class ElectronicsTax extends TaxCalculator { public double computeTax(double p){return
p*0.18;} }
class LuxuryTax extends TaxCalculator { public double computeTax(double p){return p*0.28;} }
class Product {
double price; TaxCalculator calc;
Product(double p, TaxCalculator c){price=p;calc=c;}
double tax(){ return calc.computeTax(price); }
}
public class TaxDemo {
public static void main(String[] args){
Product[] cart={new Product(100,new FoodTax()),
new Product(50,new ElectronicsTax()),new Product(200,new LuxuryTax())};
double total=0;
for(Product p:cart){System.out.println(p.tax()); total+=p.tax();}
System.out.println("Total tax: "+total);
}
}
```

OUTPUT

```
5.0
9.0
56.0
Total tax: 70.0
```

Q0
6

Strategy Pattern — Runtime Behaviour Swap

*Gaming Studio · Design Patterns***MODERATE**

LOGIC / APPROACH

Interface `AttackStrategy` declares `attack()`. Each strategy (`SwordAttack`, `MagicAttack`, `ArcherAttack`) implements it. `Character` holds an `AttackStrategy` reference and delegates `attack()` to it. `setStrategy()` swaps the behaviour at runtime. No if/switch is needed — this is the Open-Closed Principle in action.

JAVA CODE

```

interface AttackStrategy { void attack(); }
class SwordAttack implements AttackStrategy {
public void attack(){ System.out.println("Sword slash!"); }
}
class MagicAttack implements AttackStrategy {
public void attack(){ System.out.println("Magic bolt fired!"); }
}
class ArcherAttack implements AttackStrategy {
public void attack(){ System.out.println("Arrow loosed!"); }
}
class Character {
private AttackStrategy strategy;
public void setStrategy(AttackStrategy s){ strategy=s; }
public void attack(){ strategy.attack(); }
}
public class StrategyDemo {
public static void main(String[] args){
Character hero=new Character();
hero.setStrategy(new SwordAttack()); hero.attack();
hero.setStrategy(new MagicAttack()); hero.attack();
hero.setStrategy(new ArcherAttack()); hero.attack();
}
}

```

OUTPUT

```

Sword slash!
Magic bolt fired!
Arrow loosed!

```

Q0
7

Chain of Responsibility — Request Pipeline

Netflix · Design Patterns

MODERATE

LOGIC / APPROACH

Abstract Handler has a 'next' reference and a setNext() method. Each concrete handler (Auth, RateLimit, Log, Business) checks if it can handle the request; if not, passes to next. The chain is assembled by the caller. Adding new handlers requires no modification to existing ones.

JAVA CODE

```

class Request { String token; boolean rateOk; Request(String t,boolean r){token=t;rateOk=r;} }
abstract class Handler {
protected Handler next;
public Handler setNext(Handler h){ next=h; return h; }
abstract void handle(Request r);
}
class AuthHandler extends Handler {
public void handle(Request r){
if(!r.token.equals("valid")){System.out.println("401 Unauthorized");return;}
System.out.println("Auth OK"); if(next!=null) next.handle(r);
}
}
class RateLimitHandler extends Handler {
public void handle(Request r){
if(!r.rateOk){System.out.println("429 Rate Limited");return;}
System.out.println("Rate OK"); if(next!=null) next.handle(r);
}
}
class BusinessHandler extends Handler {
public void handle(Request r){ System.out.println("200 Response"); }
}
public class ChainDemo {
public static void main(String[] args){
AuthHandler auth=new AuthHandler();
auth.setNext(new RateLimitHandler()).setNext(new BusinessHandler());
auth.handle(new Request("valid",true));
System.out.println("---");
auth.handle(new Request("bad",true));
}
}

```

OUTPUT

```

Auth OK
Rate OK
200 Response
---
401 Unauthorized

```

Q0
8

Template Method Pattern — Report Generation

Accenture · Design Patterns

MODERATE

LOGIC / APPROACH

Abstract ReportGenerator declares a final generate() method that calls: fetchData(), processData(), formatOutput(), deliver() in order. fetchData() and processData() are implemented in the base class. formatOutput() and deliver() are abstract — subclasses (PDFReport, ExcelReport) provide their own implementations. The algorithm skeleton is fixed.

JAVA CODE

```

abstract class ReportGenerator {
    final void generate(){
        fetchData(); processData(); formatOutput(); deliver();
    }
    void fetchData(){ System.out.println("Fetching data..."); }
    void processData(){ System.out.println("Processing data..."); }
    abstract void formatOutput();
    abstract void deliver();
}
class PDFReport extends ReportGenerator {
    public void formatOutput(){ System.out.println("Format as PDF"); }
    public void deliver(){ System.out.println("Deliver via email"); }
}
class ExcelReport extends ReportGenerator {
    public void formatOutput(){ System.out.println("Format as Excel"); }
    public void deliver(){ System.out.println("Deliver via FTP"); }
}
public class TemplateDemo {
    public static void main(String[] args){
        new PDFReport().generate();
        System.out.println("---");
        new ExcelReport().generate();
    }
}

```

OUTPUT

```

Fetching data...
Processing data...
Format as PDF
Deliver via email
---
Fetching data...
Processing data...
Format as Excel
Deliver via FTP

```

Q0
9

Visitor Pattern — AST Double Dispatch

Palantir · Compiler Design

VERY HARD

LOGIC / APPROACH

Interface Node has accept(Visitor v). Each node class (NumNode, AddNode, MulNode) implements accept() by calling v.visit(this) — this is the first dispatch (on Node type). Visitor interface has overloaded visit() for each node type — the second dispatch. EvalVisitor computes values; PrettyPrintVisitor prints infix notation. Zero instance of checks needed.

JAVA CODE

```

interface Visitor<T> { T visit(NumNode n); T visit(AddNode n); T visit(MulNode n); }
interface Node { <T> T accept(Visitor<T> v); }
class NumNode implements Node { int val; NumNode(int v){val=v;}
public <T> T accept(Visitor<T> v){ return v.visit(this); } }
class AddNode implements Node { Node l,r; AddNode(Node l,Node r){this.l=l;this.r=r;}
public <T> T accept(Visitor<T> v){ return v.visit(this); } }
class MulNode implements Node { Node l,r; MulNode(Node l,Node r){this.l=l;this.r=r;}
public <T> T accept(Visitor<T> v){ return v.visit(this); } }
class EvalVisitor implements Visitor<Integer> {
public Integer visit(NumNode n){ return n.val; }
public Integer visit(AddNode n){ return n.l.accept(this)+n.r.accept(this); }
public Integer visit(MulNode n){ return n.l.accept(this)*n.r.accept(this); }
}
class PrettyPrint implements Visitor<String> {
public String visit(NumNode n){ return String.valueOf(n.val); }
public String visit(AddNode n){ return "("+n.l.accept(this)+"+"+n.r.accept(this)+")"; }
public String visit(MulNode n){ return n.l.accept(this)+"*"+n.r.accept(this); }
}
public class VisitorDemo {
public static void main(String[] args){
Node ast=new MulNode(new NumNode(2),new AddNode(new NumNode(3),new NumNode(4)));
System.out.println(ast.accept(new EvalVisitor()));
System.out.println(ast.accept(new PrettyPrint()));
}
}

```

OUTPUT

```

14
2*(3+4)

```

Q1
0

JVM vtable — Megamorphic Call Site Benchmark

Azul Systems · JVM Performance

VERY HARD

LOGIC / APPROACH

Monomorphic call sites (one concrete type) allow the JIT to inline the method body — ~0.5ns/call. Bimorphic (two types) allows conditional inlining. Megamorphic (3+ types) forces true vtable dispatch with no inlining — ~4ns/call. Sealed classes with final subclasses give the JIT type certainty, enabling re-monomorphisation.

JAVA CODE

```
// Requires JMH: org.openjdk.jmh
import org.openjdk.jmh.annotations.*;
import java.util.concurrent.TimeUnit;
public sealed interface Shape permits Circ, Rect, Tri { double area(); }
final class Circ implements Shape { double r; Circ(double r){this.r=r;}
public double area(){ return Math.PI*r*r; } }
final class Rect implements Shape { double w,h; Rect(double w,double h){this.w=w;this.h=h;}
public double area(){ return w*h; } }
final class Tri implements Shape { double b,h; Tri(double b,double h){this.b=b;this.h=h;}
public double area(){ return 0.5*b*h; } }
@BenchmarkMode(Mode.AverageTime) @OutputTimeUnit(TimeUnit.NANOSECONDS)
@State(Scope.Thread)
public class VTableBench {
Shape mono = new Circ(1);
Shape[] mega = {new Circ(1),new Rect(1,1),new Tri(1,1)};
@Benchmark public double monomorphic(){ return mono.area(); }
@Benchmark public double megamorphic(){
double s=0; for(Shape sh:mega) s+=sh.area(); return s;
}
}
```

OUTPUT

```
monomorphic: ~0.5 ns/op (JIT inlines)
megamorphic: ~4.0 ns/op (vtable dispatch)
```